- You can configure multiple `ILoggerProvider` instances in ASP.NET Core, which define where logs are output. `WebApplicationBuilder` adds the console and debug providers, and you can add providers using the `Logging` property.
- You can control logging output verbosity using configuration. `WebApplicationBuilder` uses the `"Logging"` configuration section to control output verbosity. You typically filter out more logs in production than when developing your application.
- Only a single log filtering rule is selected for each logging provider when determining whether to output a log message. The most specific rule is selected based on the logging provider and the category of the log message.
- Structured logging involves recording logs so that they can be easily queried and filtered, instead of the default unstructured format that’s output to the console. This makes analyzing logs, searching for problems, and identifying patterns easier.
- You can add properties to a structured log by using scope blocks. A scope block is created by calling `ILogger.BeginScope<T>(state)` in a `using` block. The state can be any object and is added to all log messages inside the scope block.